

Similarity Search by Hand

Estimated time needed: **30** minutes

In this lab, you will learn how to manually compute several common distance and similarity metrics used in machine learning and information retrieval. These metrics are fundamental to similarity search, which involves comparing data points—often represented as embedding vectors—to identify those that are most alike. Embedding vectors encode information such that their direction typically reflects the semantic content or topic of the input (e.g., a piece of text), while their magnitude can represent the intensity or importance of the encoded features. In this lab, you will calculate the Euclidean distance (L2 norm), dot product, and cosine similarity, gaining a deeper understanding of how these metrics quantify similarity and power search algorithms behind the scenes.

Table of Contents

1. Similarity Search by Hand
    A. Objectives
    B. Setup
        a. Install Required Libraries
        b. Restart the Kernel
        c. Import Required Libraries
    C. Obtain Vector Embeddings
    D. L2 (Euclidean) Distance
        a. Manual implementation of L2 distance calculation
            i. Exercise 1 - Make the manual calculation more efficient
        b. Calculate L2 distance using `scipy`
        c. Interpret the L2 Distance Results
    E. Dot Product Similarity and Distance
        a. Manual implementation of dot product calculation
        b. Calculate the dot product using matrix multiplication
        c. Calculate dot product distance
    F. Cosine Similarity and Distance
        a. Manual implementation of cosine similarity calculation
            i. Calculate the L2 norm
            ii. Normalize embedding vectors
                1. Exercise 2 - Verify that vectors are normalized
                2. Normalize embeddings using PyTorch

iii. Calculate cosine similarity manually
b. Calculate cosine similarity using matrix multiplication
c. Calculate cosine distance
2. Exercise 3 - Similarity Search Using a Query
3. Wrap-up
4. Authors

Objectives

After completing this lab you will be able to:

- **Compute vector similarities and distances**, including:
  - **L2 (Euclidean) distance**
  - **Dot product similarity and distance**
  - **Cosine similarity and distance**
- **Implement similarity calculations manually** using custom Python functions to reinforce core concepts.
- **Use built-in functions and methods** from external Python libraries (such as NumPy and PyTorch) to perform these computations efficiently.
- **Apply similarity measures to perform a basic similarity search**, comparing pairs of vectors to identify the most similar ones.

Setup

Install Required Libraries

Run the cell below to install the required libraries for this lab.

> **Note**: This process may take 3–5 minutes to complete.

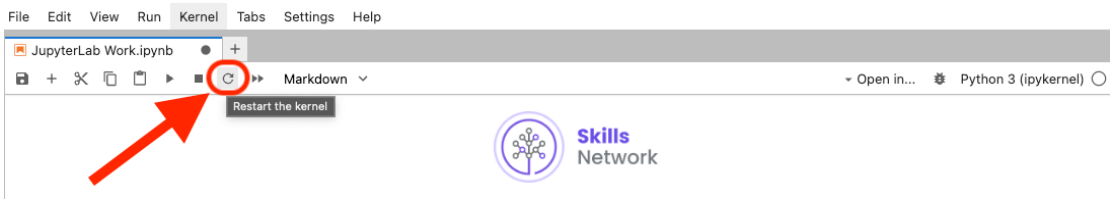
In [1]:
```
!pip install sentence-transformers==4.1.0 | tail -n 1
```

```
Successfully installed Pillow-12.3.0 cuda-bindings-13.3.1 cuda-pathfinde
r-1.5.6 cuda-toolkit-13.0.3.0 filelock-3.31.1 fsspec-2026.6.0 hf-xet-1.5.2
huggingface-hub-0.36.2 joblib-1.5.3 mpmath-1.3.0 narwhals-2.24.0 network
x-3.6.1 numpy-2.5.1 nvidia-cublas-13.1.1.3 nvidia-cuda-cupti-13.0.85 nvidi
a-cuda-nvrtc-13.0.88 nvidia-cuda-runtime-13.0.96 nvidia-cudnn-cu13-9.2
0.0.48 nvidia-cufft-12.0.0.61 nvidia-cufile-1.15.1.6 nvidia-curand-1
0.4.0.35 nvidia-cusolver-12.0.4.66 nvidia-cusparse-12.6.3.3 nvidia-cuspars
elt-cu13-0.8.1 nvidia-nccl-cu13-2.29.7 nvidia-nvjitlink-13.3.33 nvidia-nvs
hmem-cu13-3.4.5 nvidia-nvtx-13.0.85 regex-2026.7.19 safetensors-0.8.0 scik
it-learn-1.9.0 scipy-1.18.0 sentence-transformers-4.1.0 setuptools-83.0.0
sympy-1.14.0 threadpoolctl-3.6.0 tokenizers-0.22.2 torch-2.13.0 transforme
rs-4.57.6 triton-3.7.1
```

Restart the Kernel

After the above cell finishes running and successfully installs required libraries, please restart the kernel by clicking on the `Restart the kernel` button as indicated on the image below, or by going to `Kernel` --> `Restart kernel...` in the file menu.

> **Note**: You only need to restart the kernel once. After restarting, do not re-run this cell or any of the previous ones. Instead, continue with the lab starting from the next section, Import Required Libraries.



Import Required Libraries

For this lab, we will be using the following external libraries:

- `numpy` for mathematical operations.
- `scipy` for additional mathematical operations not found in `numpy`.
- `torch` for vector operations, although this library is also typically used for deep learning.
- `sentence_transformers` for obtaining vector embeddings from text data using pre-trained models.

Run the cell below to import these libraries, along with the built-in `math` module from Python's standard library:

In [2]:
```
import math

import numpy as np
import scipy
import torch
from sentence_transformers import SentenceTransformer
```

Obtain Vector Embeddings

To calculate distance and similarity metrics, we first need to generate vector embeddings for some text documents. Let's begin by defining a few example documents:

In [3]:
```
# Example documents
documents = [
    'Bugs introduced by the intern had to be squashed by the lead develop
    'Bugs found by the quality assurance engineer were difficult to debug
    'Bugs are common throughout the warm summer months, according to the
```

```
    'Bugs, in particular spiders, are extensively studied by arachnologis
]
```

In [4]:
```
# Load a pre-trained model
model = SentenceTransformer('paraphrase-MiniLM-L6-v2')
```

```
modules.json:   0%|          | 0.00/229 [00:00<?, ?B/s]
config_sentence_transformers.json:   0%|          | 0.00/122 [00:00<?, ?B/
s]
README.md: 0.00B [00:00, ?B/s]
sentence_bert_config.json:   0%|          | 0.00/53.0 [00:00<?, ?B/s]
config.json:   0%|          | 0.00/629 [00:00<?, ?B/s]
model.safetensors:   0%|          | 0.00/90.9M [00:00<?, ?B/s]
tokenizer_config.json:   0%|          | 0.00/314 [00:00<?, ?B/s]
vocab.txt: 0.00B [00:00, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
special_tokens_map.json:   0%|          | 0.00/112 [00:00<?, ?B/s]
config.json:   0%|          | 0.00/190 [00:00<?, ?B/s]
```

The code above creates an instance of the SentenceTransformer class from the sentence_transformers library, which is commonly used to generate vector embeddings from pre-trained models.

In this example, we're using the paraphrase-MiniLM-L6-v2 model. It was trained on pairs of paraphrased sentences, with the goal of generating similar embeddings for sentences that express the same meaning. While the model was originally designed for paraphrase identification, it also performs well on general semantic similarity tasks—like the ones we're exploring in this lab.

Let's now use the model instance to encode the documents into embedding vectors:

In [5]:
```
# Generate embeddings
embeddings = model.encode(documents)
```

Let's explore the shape of our embeddings:

In [6]:
```
embeddings.shape
```

Out[6]:
```
(4, 384)
```

And let's have a quick look at them:

In [7]:
```
embeddings
```

Out[7]:
```
array([[-0.22804348, -0.24647684, -0.00319294, ...,  0.45528147,
         0.6341976 ,  0.5375049 ],
       [-0.35791597, -0.32083988,  0.15963267, ..., -0.07050666,
         0.92750263,  0.34377286],
       [ 0.20302926, -0.26898623,  0.1628511 , ..., -0.19650966,
        -0.03379833,  0.5956154 ],
       [-0.04264304, -0.45721593, -0.09526499, ..., -0.5803074 ,
         0.17248419,  0.09127873]], shape=(4, 384), dtype=float32)
```

As shown above, the four text documents have been converted into a NumPy array with shape (4, 384). Each row in the array represents the embedding of one document, and the number 384 is the dimensionality of the embeddings produced by the paraphrase-

MiniLM-L6-v2 model. In other words, each document has been transformed into a numerical vector containing 384 values.

L2 (Euclidean) Distance

The L2 (Euclidean) distance between 2 vectors a and b can be calculated using $$ \text{L2}(a,b) = \ \sqrt{\sum_{i=1}^n (a_i - b_i)^2} $$

Let's try to implement the L2 distance manually before looking at off-the-shelf solutions available from third party libraries.

Manual implementation of L2 distance calculation

The function below implements the L2 distance formula. It first calculates the sum of the squared differences between corresponding elements, then returns the square root of that sum:

In [8]:
```
def euclidean_distance_fn(vector1, vector2):
    squared_sum = sum((x - y) ** 2 for x, y in zip(vector1, vector2))
    return math.sqrt(squared_sum)
```

Note that `euclidean_distance_fn` computes the distance between two individual vectors, not across an entire array. To see how it works, let's calculate the distance between the first vector (index 0) and the second vector (index 1):

In [9]:
```
euclidean_distance_fn(embeddings[0], embeddings[1])
```

Out[9]:
```
5.961789851413909
```

Of course, it does not matter which vector we put first into the function, the resulting distance is the same:

In [10]:
```
euclidean_distance_fn(embeddings[1], embeddings[0])
```

Out[10]:
```
5.961789851413909
```

In order to calculate all of the distances between all of the vectors in the `embeddings` array, we can use nested loops. In the following, `i` and `j` are the indices of the vectors in the array:

In [11]:
```
l2_dist_manual = np.zeros([4,4])
for i in range(embeddings.shape[0]):
    for j in range(embeddings.shape[0]):
        l2_dist_manual[i,j] = euclidean_distance_fn(embeddings[i], embedd

l2_dist_manual
```

Out[11]:
```
array([[0.        , 5.96178985, 7.33939781, 7.15578276],
       [5.96178985, 0.        , 7.76861601, 7.39359048],
       [7.33939781, 7.76861601, 0.        , 5.91992735],
       [7.15578276, 7.39359048, 5.91992735, 0.        ]])
```

`l2_dist_manual` is a 4×4 array where each element represents the L2 distance between two vectors: the vector at the row index and the vector at the column index. For example, the distance between the first vector (index 0) and the second vector (index 1) is located at position `[0, 1]` in the array:

In [12]:
```
l2_dist_manual[0,1]
```

Out[12]:
```
np.float64(5.961789851413909)
```

Moreover, you'll notice that the upper and lower triangles of the array are identical. This is not a coincidence. Just like with the `euclidean_distance_fn` function, the distance between vectors a and b should be identical to the distance between vectors b and a:

In [ ]:
```
l2_dist_manual[1,0]
```

Exercise 1 - Make the manual calculation more efficient

The code used to populate the `l2_dist_manual array` is not very efficient. First, it redundantly calculates the distance between a vector and itself, even though the L2 distance in such cases is always zero. Second, the array is symmetric—meaning the distance between vectors at indices i and j is the same as between j and i. Therefore, each distance only needs to be computed once.

In the cell below, write an improved version of the code that avoids these inefficiencies. You only need to modify the part that says `### YOUR CODE GOES HERE ###`

In [13]:
```
l2_dist_manual_improved = np.zeros([4,4])
for i in range(embeddings.shape[0]):
    for j in range(embeddings.shape[0]):
        ### YOUR CODE GOES HERE ###
        if i > j:
            l2_dist_manual_improved[i,j] = euclidean_distance_fn(embeddin
        else:
            l2_dist_manual_improved[i,j] = l2_dist_manual[j,i]

l2_dist_manual_improved
```

Out[13]:
```
array([[0.        , 5.96178985, 7.33939781, 7.15578276],
       [5.96178985, 0.        , 7.76861601, 7.39359048],
       [7.33939781, 7.76861601, 0.        , 5.91992735],
       [7.15578276, 7.39359048, 5.91992735, 0.        ]])
```

▶ Click here for the solution

Calculate L2 distance using `scipy`

Instead of writing your own function, you can use a variety of different functions provided by external libraries to calculate the L2 distance. The example below uses a function from `scipy` :

In [14]:
```
l2_dist_scipy = scipy.spatial.distance.cdist(embeddings, embeddings, 'euc
l2_dist_scipy
```

Out[14]:
```
array([[0.        , 5.96178961, 7.33939853, 7.15578165],
       [5.96178961, 0.        , 7.76861598, 7.39359146],
       [7.33939853, 7.76861598, 0.        , 5.9199269 ],
       [7.15578165, 7.39359146, 5.9199269 , 0.        ]])
```

The following verifies that `l2_dist_manual` and `l2_dist_scipy` are identical (after accounting for rounding errors) by using the `allclose()` function from `numpy`:

In [15]:
```
np.allclose(l2_dist_manual, l2_dist_scipy)
```

Out[15]:
```
True
```

Interpret the L2 Distance Results

An analysis of `l2_dist_scipy` shows that, in this case, the L2 distance metric performed well for similarity search. For example, the first vector—corresponding to the sentence `Bugs introduced by the intern had to be squashed by the lead developer.` —had the smallest distance to the second vector, which represents the sentence `Bugs found by the quality assurance engineer were difficult to debug.` This result aligns with expectations, as both sentences refer to programming bugs.

Similarly, the third and fourth sentences—both related to physical bugs rather than programming—were closest to each other in terms of distance, which again matches our intuition.

Dot Product Similarity and Distance

The dot product between vectors a and b is calculated using
$$ a \cdot b = \sum_{i=1}^{n} a_i b_i $$

Let's define a custom function for calculating the dot product between two vectors.

Manual implementation of dot product calculation

The function below implements the dot product formula:

In [16]:
```
def dot_product_fn(vector1, vector2):
    return sum(x * y for x, y in zip(vector1, vector2))
```

Let's use the dot product function to calculate the dot product between the first and second vectors:

In [17]:
```
dot_product_fn(embeddings[0], embeddings[1])
```

Out[17]:
```
np.float32(18.535397)
```

Just like we did with the L2 distance calculation, let's use nested loops to calculate the dot product between all vectors in the `embeddings` array:

In [18]:
```
dot_product_manual = np.empty([4,4])
for i in range(embeddings.shape[0]):
    for j in range(embeddings.shape[0]):
        dot_product_manual[i,j] = dot_product_fn(embeddings[i], embedding

dot_product_manual
```

Out[18]:
```
array([[33.74439621, 18.53539658,  8.56981564,  7.83093357],
       [18.53539658, 38.86933136,  7.88997412,  8.66340351],
       [ 8.56981564,  7.88997412, 37.26199722, 17.66957474],
       [ 7.83093357,  8.66340351, 17.66957474, 33.12267303]])
```

First, observe that the diagonal of the `dot_product_manual matrix` contains non-zero values. This is expected, as the dot product of a vector with itself is not zero. However, since these self-similarity scores aren't particularly informative, we can safely ignore them.

Excluding the diagonal, the matrix is symmetric—the upper triangle mirrors the lower triangle—as was the case with the L2 distance.

It's also important to remember that the dot product measures similarity, not distance. Higher values indicate greater similarity between vectors.

In this case, the dot product similarity performed well: the first two sentences are most similar to each other, and the last two sentences are most similar to each other, which aligns with our expectations.

Calculate the dot product using matrix multiplication

We can compute the dot product efficiently using the matrix multiplication operator `@`. To do this, we multiply the `embeddings` matrix by its transpose. Since `embeddings` has a shape of 4×384, its transpose will be 384×4. Multiplying these gives us a 4×4 matrix, which is the desired result:

In [19]:
```
# Matrix multiplication operator
dot_product_operator = embeddings @ embeddings.T
dot_product_operator
```

Out[19]:
```
array([[33.744396 , 18.535397 ,  8.569815 ,  7.8309336],
       [18.535397 , 38.869335 ,  7.889973 ,  8.6634035],
       [ 8.569815 ,  7.889973 , 37.261997 , 17.669575 ],
       [ 7.8309336,  8.6634035, 17.669575 , 33.122677 ]], dtype=float32)
```

We can verify that the matrix multiplication operator returns the same result as our custom function after accounting for rounding:

In [20]:
```
np.allclose(dot_product_manual, dot_product_operator, atol=1e-05)
```

Out[20]:
```
True
```

Equivalently, if both of the matrices we want to multiply are two-dimensional, we can use the `matmul()` function from `numpy` instead:

In [21]:
```
# Equivalent to `np.matmul()` if both arrays are 2-D:
np.matmul(embeddings,embeddings.T)
```

Out[21]:
```
array([[33.744396 , 18.535397 ,  8.569815 ,  7.8309336],
       [18.535397 , 38.869335 ,  7.889973 ,  8.6634035],
       [ 8.569815 ,  7.889973 , 37.261997 , 17.669575 ],
       [ 7.8309336,  8.6634035, 17.669575 , 33.122677 ]], dtype=float32)
```

Finally, `numpy` also provides a `dot()` function, which provides an identical result:

In [22]:
```
# `np.dot` returns an identical result, but `np.matmul` is recommended if
np.dot(embeddings,embeddings.T)
```

Out[22]:
```
array([[33.744396 , 18.535397 ,  8.569815 ,  7.8309336],
       [18.535397 , 38.869335 ,  7.889973 ,  8.6634035],
       [ 8.569815 ,  7.889973 , 37.261997 , 17.669575 ],
       [ 7.8309336,  8.6634035, 17.669575 , 33.122677 ]], dtype=float32)
```

Calculate dot product distance

The dot product between two vectors provides a similarity score. If, on the other hand, we would like a distance, we can simply take the negative of the dot product:

In [23]:
```
dot_product_distance = -dot_product_manual
dot_product_distance
```

Out[23]:
```
array([[-33.74439621, -18.53539658,  -8.56981564,  -7.83093357],
       [-18.53539658, -38.86933136,  -7.88997412,  -8.66340351],
       [ -8.56981564,  -7.88997412, -37.26199722, -17.66957474],
       [ -7.83093357,  -8.66340351, -17.66957474, -33.12267303]])
```

Although it might seem unusual for all the distances to be negative, the essential property of a distance metric is still preserved: smaller values indicate lower distance and thus greater similarity. So even with negative values, the relative comparisons remain valid—lower values still correspond to shorter distances.

Cosine Similarity and Distance

The cosine similarity between vectors a and b is calculated using
$$ \text{cossim}(a, b) = \frac{a \cdot b}{||a|| \ ||b||} = \frac{a}{||a||} \cdot \frac{b}{||b||} $$

where $||a|| = \sqrt{\sum_{k=1}^n a_k^2}$ is the L2 norm, or the magnitude, of vector a, and $\cdot$ is the dot product.

Also note that $\frac{a}{||a||}$ represents a normalized vector. This means it has the same direction as vector a but a magnitude (or length) of 1. Thus, cosine similarity can be calculated by taking the dot product of two normalized vectors.

Manual implementation of cosine similarity calculation

Since we've already covered how to compute the dot product, our strategy for manually calculating cosine similarity will focus on normalizing the vectors. This is because cosine similarity is simply the dot product of two normalized vectors, as was shown after the last equals sign in the cosine similarity calculation formula.

However, in order to normalize vectors, we must first compute their L2 norms.

Calculate the L2 norm

The following calculates the L2 norms for all the vectors in the `embeddings` array. The calculation simply squares each vector component, sums across columns (note the `axis=1` parameter in the `sum`), and takes a square root:

In [24]:
```
# L2 norms
l2_norms = np.sqrt(np.sum(embeddings**2, axis=1))
l2_norms
```

Out[24]:
```
array([5.808994 , 6.2345276, 6.1042604, 5.75523  ], dtype=float32)
```

Note that the result is a vector with 4 numbers, each of which corresponds to the L2 norm, or the magnitude, of each vector. In order to normalize the vectors to a lenght of one, we should divide each vector's components by the norm. However, in order to do that efficiently, let's reshape the `l2_norms` vector into a 4x1 array:

In [25]:
```
# L2 norms reshaped
l2_norms_reshaped = l2_norms.reshape(-1,1)
l2_norms_reshaped
```

Out[25]:
```
array([[5.808994 ],
       [6.2345276],
       [6.1042604],
       [5.75523  ]], dtype=float32)
```

Normalize embedding vectors

The following code calculates normalized embedding vectors by dividing every component in the vector by the vector's L2 norm:

In [26]:
```
normalized_embeddings_manual = embeddings/l2_norms_reshaped
normalized_embeddings_manual
```

Out[26]:
```
array([[-0.03925697, -0.04243021, -0.00054966, ...,  0.07837527,
         0.10917512,  0.09252978],
       [-0.05740868, -0.05146178,  0.02560461, ..., -0.01130906,
         0.14876871,  0.05514016],
       [ 0.03326026, -0.04406533,  0.02667827, ..., -0.03219222,
        -0.00553684,  0.09757372],
       [-0.00740944, -0.07944356, -0.01655277, ..., -0.10083131,
         0.02996999,  0.01586014]], shape=(4, 384), dtype=float32)
```

Exercise 2 - Verify that vectors are normalized

Verify that `normalized_embeddings_manual` are normalized vectors by making sure that the length of each vector is equal to 1.

In [27]:
```
### YOUR CODE GOES HERE ###
np.sqrt(np.sum(normalized_embeddings_manual**2, axis=1))
```

Out[27]:
```
array([0.99999994, 0.99999994, 1.        , 1.        ], dtype=float32)
```

▶ Click here for the solution

Normalize embeddings using PyTorch

You can normalize embeddings in PyTorch using `torch.nn.functional.normalize()`. If your data is in a NumPy array, first convert it to a PyTorch tensor using `torch.from_numpy()`. After normalization, convert the tensor back to a NumPy array using the `numpy()` method:

In [28]:
```
normalized_embeddings_torch = torch.nn.functional.normalize(
    torch.from_numpy(embeddings)
).numpy()
normalized_embeddings_torch
```

Out[28]:
```
array([[-0.03925697, -0.04243021, -0.00054966, ...,  0.07837527,
         0.10917512,  0.09252978],
       [-0.05740868, -0.05146178,  0.02560461, ..., -0.01130906,
         0.14876871,  0.05514016],
       [ 0.03326026, -0.04406533,  0.02667827, ..., -0.03219222,
        -0.00553684,  0.09757372],
       [-0.00740944, -0.07944355, -0.01655277, ..., -0.10083131,
         0.02996999,  0.01586013]], shape=(4, 384), dtype=float32)
```

We can verify that the normalized embeddings we calculated manually and the normalized embeddings calculated using `torch` are indeed identical using numpy's `allclose()` function:

In [29]:
```
np.allclose(normalized_embeddings_manual, normalized_embeddings_torch)
```

Out[29]:
```
True
```

Calculate cosine similarity manually

To calculate cosine similarity between two normalized embedding vectors, we simply take their dot product. To do this, we can leverage the dot product function we defined before. For instance, the following calculates the cosine similarity between the vector embeddings of the first and second sentence:

In [30]:
```
dot_product_fn(normalized_embeddings_manual[0], normalized_embeddings_man
```

Out[30]:
```
np.float32(0.51179683)
```

Likewise, to calculate the cosine similarities between all normalized vectors, we can use

a nested loop:

In [31]:
```
cosine_similarity_manual = np.empty([4,4])
for i in range(normalized_embeddings_manual.shape[0]):
    for j in range(normalized_embeddings_manual.shape[0]):
        cosine_similarity_manual[i,j] = dot_product_fn(
            normalized_embeddings_manual[i],
            normalized_embeddings_manual[j]
        )

cosine_similarity_manual
```

Out[31]:
```
array([[1.00000012, 0.51179683, 0.24167828, 0.23423405],
       [0.51179683, 1.00000024, 0.20731883, 0.24144743],
       [0.24167828, 0.20731883, 1.00000095, 0.50295621],
       [0.23423405, 0.24144743, 0.50295621, 1.00000024]])
```

Cosine similarity ranges from -1 to 1. The cosine similarity matrix is symmetric, just like the matrices for L2 distance and the dot product. In this example, cosine similarity performed well: the first two sentences were most similar to each other, as were the last two—matching our expectations.

Calculate cosine similarity using matrix multiplication

Just like with the dot product, we can compute cosine similarity using matrix algebra. By multiplying the matrix of normalized embeddings with its transpose using the matrix multiplication operator, we obtain the cosine similarity matrix. This works because, once vectors are normalized, cosine similarity can be calculated by simply taking the dot product:

In [32]:
```
cosine_similarity_operator = normalized_embeddings_manual @ normalized_em
cosine_similarity_operator
```

Out[32]:
```
array([[1.0000001 , 0.51179683, 0.24167825, 0.23423405],
       [0.51179683, 1.0000002 , 0.20731883, 0.24144743],
       [0.24167825, 0.20731883, 1.000001  , 0.50295615],
       [0.23423405, 0.24144743, 0.50295615, 1.0000002 ]], dtype=float32)
```

We can verify that the matrix algebra solution is the same as the one found using the nested loop:

In [33]:
```
np.allclose(cosine_similarity_manual, cosine_similarity_operator)
```

Out[33]:
```
True
```

Calculate cosine distance

The cosine distance between vectors a and b is simply 1 minus the cosine similarity between a and b:
$$ 1 - cossim(a,b) $$

Using numpy, this can be calculated as follows:

In [34]:
```
1 - cosine_similarity_manual
```

Out[34]:
```
array([[-1.19209290e-07,  4.88203168e-01,  7.58321717e-01,
         7.65765950e-01],
       [ 4.88203168e-01, -2.38418579e-07,  7.92681172e-01,
         7.58552566e-01],
       [ 7.58321717e-01,  7.92681172e-01, -9.53674316e-07,
         4.97043788e-01],
       [ 7.65765950e-01,  7.58552566e-01,  4.97043788e-01,
        -2.38418579e-07]])
```

Exercise 3 - Similarity Search Using a Query

In the above examples, we calculated similarity between 4 documents:

```
documents = [
    'Bugs introduced by the intern had to be squashed by the lead developer.',
    'Bugs found by the quality assurance engineer were difficult to debug.',
    'Bugs are common throughout the warm summer months, according to the entomologist.',
    'Bugs, in particular spiders, are extensively studied by arachnologists.'
]
```

Now, your task is to find which of these 4 documents is most similar to the query `Who is responsible for a coding project and fixing others' mistakes?` using cosine similarity. You can reuse the `documents` and `normalized_embeddings_manual` arrays in your answer:

In [35]:
```
### YOUR CODE GOES HERE ###
query_embedding = model.encode(
    ["Who is responsible for a coding project and fixing others' mistakes"]
)
normalized_embedding = torch.nn.functional.normalize(
    torch.from_numpy(query_embedding)
).numpy()
```

▶ Click here for the solution

Wrap-up

In this lab, you explored how to compute L2 distance, dot product similarity and distance, as well as cosine similarity and distance between pairs of vectors for the purposes of performing a similarity search. These computations were performed both through manually defined functions and by using built-in functions and methods from external Python libraries. This helped illustrate the mathematical foundations of vector similarity and how they are implemented in practical coding scenarios.

Authors

Wojciech "Victor" Fulmyk

Change Log

▸ Click here for the changelog

Copyright © IBM Corporation. All rights reserved.

As shown above, there are four example documents, each consisting of a single sentence. These sentences are intentionally designed to be challenging for semantic similarity search.

All four sentences begin with the word "Bugs," but they refer to different meanings of the word depending on context. The first two sentences relate to software bugs in programming, while the last two refer to physical bugs, such as insects or spiders.

The key to distinguishing between these meanings lies in the context, particularly the type of professional mentioned in each sentence. For example, if we replaced the word "arachnologists" (scientists who study spiders and other arthropods) in the last sentence with "lead developers," the sentence would instead refer to programming bugs.

Let's now define a model that will embed these text documents into numerical vectors: